

SOFTWARE REQUIREMENTS SPECIFICATION

GateSphere Platform

Enterprise Residential Community, Visitor, Security & Facility Management System

Project Codename: GateSphere Enterprise (GSE-2026)

Document Type: Software Requirements Specification (SRS)

Prepared For: VPD Technologies Pvt. Ltd. (Implementation Vendor)

Independent QA Partner: Sivion Global Technologies

Source Documents: GateSphere PRD (10 pages) & GateSphere Functionality and Module Workflows

Delivery Commitment: 30 calendar days from officially approved project start date

Technology Stack: Frontend: Next.js | Backend: FastAPI | Database: PostgreSQL

Document Status: Draft v1.1 — revised technology stack & week-wise delivery schedule

Date of Issuance: August 2026

Confidential — Enterprise Engineering Contract

Revision Note (v1.1)

*This revision updates the technology stack and delivery-schedule presentation from Draft v1.0. The backend framework is standardized on **FastAPI**. The frontend framework is standardized on **Next.js**. The 30-calendar-day delivery schedule (Section 9.1) has been restructured from a day-range milestone table into a week-wise timeline for clearer planning and tracking. No functional scope, RBAC matrix, or acceptance criteria have been altered by this revision.*

Table of Contents

1. Introduction	4
1.1 Purpose	4
1.2 Scope	4
1.3 Definitions, Acronyms and Abbreviations	4
1.4 References	5
1.5 Document Overview	5
2. Overall Description	5
2.1 Product Perspective	5
2.2 Product Functions (Summary)	6
2.3 User Classes and Characteristics	6
2.4 Operating Environment	7
2.5 Design and Implementation Constraints	7
2.6 Assumptions and Dependencies	7
3. System Features — Functional Requirements	8
FR-01 through FR-19	8
4. External Interface Requirements	13
5. Non-Functional Requirements	13
6. Data Requirements — Mandatory Seed & Synthetic Sample Data	15
7. Role-Based Access Control (RBAC) Matrix	16
8. Quality Assurance, Defect Classification & Acceptance Criteria	17
9. Delivery Schedule & Change-Control Protocol (Week-Wise)	18
10. Traceability & Interpretation Notes	19

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) defines the functional and non-functional requirements for the GateSphere Platform, an enterprise-grade residential community, visitor, security, and facility-management system. It translates the binding Project Requirement Document (PRD) and Technical Scope issued to VPD Technologies Pvt. Ltd. into a structured requirements baseline suitable for design, development, and independent QA validation by Sivion Global Technologies.

This document is intended for the development team, project management, the independent QA/testing authority, and the client stakeholders responsible for final acceptance.

1.2 Scope

GateSphere is commissioned as an original, enterprise-grade platform — functionally comparable to the category of platforms such as MyGate — featuring custom UI/UX design, micro-modular architecture, relational database schemas, RESTful APIs, and comprehensive role-based execution workflows. GateSphere is explicitly NOT a basic CRUD website or proof-of-concept prototype; it is a critical application managing physical community security, financial billing records, statutory compliance data, and live gate traffic.

The system in scope covers: multi-tenant community and property management; role-based access control; visitor lifecycle management; security gate operations; domestic staff and multi-apartment assignment; delivery management; vehicle, parking and violation tracking; maintenance billing and accounting; complaint/service-desk SLA management; amenity booking; community communication/broadcasts; emergency and security incident management; role-specific dashboards; a notification engine; search/reporting/audit logging; mandatory synthetic seed data; and the governing technical architecture, QA, and acceptance framework.

Out of scope (not specified in the source PRD and therefore excluded unless separately authorized): live payment-gateway integration (only simulated payment workflows are required), a specific SMS/WhatsApp provider (only an integration-ready mock framework is required), biometric attendance, facial recognition, GPS tracking, and external police-verification API integration.

1.3 Definitions, Acronyms and Abbreviations

Term	Definition
PRD	Project Requirement Document — the binding source specification issued to VPD.
SRS	Software Requirements Specification — this document.
RBAC	Role-Based Access Control.
VPD	VPD Technologies Pvt. Ltd. — the implementation vendor.
Sivion	Sivion Global Technologies — the independent testing & QA partner.
SLA	Service Level Agreement, applied to complaint/service-ticket resolution timelines.
JWT	JSON Web Token, used for stateless session/authentication management.
CRUD	Create, Read, Update, Delete — basic data operations; GateSphere is explicitly more than this.
Unit	An individual flat, penthouse, or villa within a tower/block.
Community	An independent gated premise or township — the top-level multi-tenancy boundary.

1.4 References

- GateSphere Platform — Project Requirement Document (PRD), Technical Scope, Development Plan, Testing Framework & Acceptance Criteria (VPD Technologies Pvt. Ltd., August 2026).

- GateSphere Functionality and Module Workflows — detailed functional interpretation of the PRD.
- IEEE 830 / ISO-IEC-IEEE 29148 conventions, used as the structural template for this SRS.

1.5 Document Overview

Section 2 describes the product perspective, user classes, and overall constraints. Section 3 specifies detailed functional requirements per module. Section 4 defines external interface requirements. Section 5 defines non-functional requirements. Section 6 defines data and seed-data requirements. Section 7 defines the RBAC matrix. Section 8 defines the QA, defect-classification, and acceptance framework. Section 9 defines the week-wise delivery schedule and change-control protocol.

2. Overall Description

2.1 Product Perspective

GateSphere is a new, standalone, multi-tenant web application (not an extension of an existing system). Its functional model is layered as follows: Communities → Towers/Blocks → Floors & Units → Resident Profiles form the master data layer; Authentication and RBAC govern every protected screen and API; and the operational modules (visitor, gate, staff, delivery, vehicle/parking, billing, complaints, amenities, communication, incidents) read and write against that shared context. Dashboards, search/reporting, and audit logging aggregate this operational data for role-specific visibility, while notifications distribute events across modules.

The following table summarizes the core inter-module dependency chain that the architecture must preserve end-to-end:

Functional Layer	Modules	Primary Relationship
Identity & Access	Authentication, RBAC, multi-tenant isolation	Controls every protected screen/API
Community Master	Communities, towers/blocks, floors, units, residents	Provides ownership/tenancy and location context
Gate Operations	Visitors, security gate, delivery, domestic staff	Controls physical entry/exit
Mobility	Vehicles, parking, violations	Connects residents/visitors to vehicles and slots
Resident Services	Maintenance billing, complaints, amenities	Provides recurring services and resident actions
Communication & Safety	Broadcasts, notifications, incidents/emergency	Communicates events and records response
Governance	Dashboards, search/reporting, audit logs	Provides visibility, compliance and traceability
Quality & Acceptance	Seed data, weekly task view, Sivion QA	Validates delivery readiness for acceptance

2.2 Product Functions (Summary)

- Multi-tenant community, tower, floor, unit, and resident-profile management with strict data segregation.
- Role-based authentication and authorization enforced at both API and frontend rendering layers.
- End-to-end visitor lifecycle management, from gate arrival through resident approval to exit.
- Live, high-speed security-gate operations dashboard for guards and supervisors.
- Domestic staff tracking with multi-apartment assignment and attendance/ratings.
- Configurable delivery protocols per delivery category.
- Vehicle, parking allocation, and violation tracking.
- Accounting-grade maintenance billing, ledgers, and simulated payments.
- SLA-driven complaint/service-desk ticketing with escalation.
- Amenity booking with concurrency/conflict prevention.
- Targeted community communication, broadcasts, polls, and event calendars.
- Emergency and security incident logging, assignment, and resolution audit trail.
- Role-specific dashboards and KPI summaries.
- Integration-ready notification engine (push, SMS mock, WhatsApp mock).
- Search, filtering, reporting, and immutable audit logging.
- Mandatory synthetic seed data across all modules to avoid empty-state screens.

2.3 User Classes and Characteristics

GateSphere serves ten distinct role classes, each with a defined scope and administrative capability set (see the full RBAC matrix in Section 7). At a summary level:

- **Super Admin** — global platform control across multiple independent communities.
- **Community Admin** — single-community management (towers, staff, broadcasts, finance configuration, resident approvals).
- **Association Committee** — governance/oversight (financial summaries, special assessments, incident review, collection audit).
- **Facility Manager** — amenity bookings, maintenance schedules, vendor contracts, common-area service desks.
- **Security Supervisor** — guard rosters, blacklist management, incident review, gate checkpoint overrides.
- **Security Guard** — live gate entry/exit processing, delivery/cab/staff verification, panic alerts.
- **Owner/Tenant** — residential unit control: visitor approval, bill payment, complaints, amenities, family/vehicle records.
- **Domestic Staff** — workforce access: assigned apartments, gate-pass check-in, personal attendance/ratings.
- **Vendor/Technician** — assigned service tickets, entry passes, work-completion proofs.
- **Auditor** — read-only compliance access to financial ledgers, audit logs, incident histories, and user-access reports.

2.4 Operating Environment

- **Frontend:** Next.js with responsive CSS.
- **Backend:** Python using FastAPI.
- **Database:** PostgreSQL, with relational integrity and automated migrations.
- **Authentication:** session-based management with JWT/session token support, per the technical scope.
- **Deployment:** a deployment platform capable of supporting weekly release/task-tracking cadence for the duration of the project.

2.5 Design and Implementation Constraints

- The complete engineering lifecycle — including testing validation, bug remediation, and final handover — must be executed within 30 calendar days of the officially approved project start date.
- Development must not proceed on an extended timeline without prior written approval; if VPD determines the scope cannot be delivered within 30 calendar days, it must, before kickoff, submit a formal email detailing technical justification, affected modules, proposed revised dates, resource allocation, and schedule impact. Starting development without such notice binds VPD to the strict 30-day commitment.
- RBAC must be enforced at both the API routing layer and the frontend component-render layer; unauthorized users must be fully blocked from restricted APIs, screens, and administrative records, including via URL manipulation.
- Passwords must use secure hashing (Argon2 or Bcrypt); SQL access must be parameterized to prevent injection; CSRF protection, rate limiting on sensitive endpoints, and file-upload validation are mandatory.
- The delivered system must include mandatory synthetic sample data; empty screens are strictly prohibited at handover.
- All work must pass through the independent Sivion QA lifecycle before any task is considered closed; VPD-internal completion alone does not constitute acceptance.

2.6 Assumptions and Dependencies

- The PRD's payment workflow requirement is satisfied by a simulated payment flow; no live payment-gateway integration is assumed.
- The notification engine is assumed to be integration-ready but may ship with SMS/WhatsApp channels mocked rather than connected to a live provider.
- Where the PRD names a capability without specifying exact UI/API implementation, the implementation team is assumed to have latitude to choose the concrete technical design while preserving the stated business behavior.
- No additional roles beyond the ten specified in the RBAC matrix are assumed to be in scope.

3. System Features — Functional Requirements

Each subsection below defines one functional module as a set of testable “the system shall” requirements, derived from the PRD and the associated module-workflow interpretation. Module identifiers (FR-01 ... FR-19) are used for traceability into the test plan and weekly task view.

FR-01 Authentication & Session Management

- The system shall authenticate users via submitted credentials before granting access to any protected resource.
- The system shall hash passwords using Argon2 or Bcrypt; plaintext passwords shall never be persisted.
- The system shall establish an authenticated session or JWT token upon successful login and resolve the user's assigned role and permitted scope.
- The system shall present each user only with the screens and actions permitted by their resolved role.
- The system shall validate every protected API request independently at the API routing layer, regardless of frontend state.
- The system shall enforce role-based rendering on the frontend so that restricted controls are never presented to unauthorized roles.
- The system shall record sensitive administrative operations in the audit log.

FR-02 Comprehensive Role-Based Access Control (RBAC)

- The system shall implement the ten defined roles: Super Admin, Community Admin, Association Committee, Facility Manager, Security Supervisor, Security Guard, Owner/Tenant, Domestic Staff, Vendor/Technician, and Auditor.
- The system shall enforce RBAC at both the API routing layer and the frontend component-render layer.
- The system shall enforce strict multi-tenant data segregation: data belonging to one community shall never be exposed to another community.
- The system shall block unauthorized API calls, screen access, and administrative-record access attempted via URL manipulation.
- The system shall grant Auditors read-only access to financial ledgers, audit logs, incident histories, and user-access reports, with no edit privileges.

FR-03 Community & Property Management

- The system shall allow Super Admins and authorized Community Admins to create and maintain independent gated communities/townships.
- The system shall maintain towers/blocks or villa clusters associated with each community.
- The system shall maintain floors and individual residential units associated with the relevant tower/block.
- The system shall maintain ownership and tenancy mappings between residents and units, including primary owners, secondary owners, active tenants, and family members.
- The system shall maintain emergency contacts and statutory move-in/move-out clearance information per resident profile.
- The system shall expose the unit/resident context to all downstream modules (visitors, vehicles, parking, billing, complaints, amenities) while preserving community-level data segregation.

FR-04 Advanced Visitor Management

- The system shall support the visitor categories: Personal Guests, Relatives, Cab/Taxi Drivers, Delivery Executives, Service Technicians, Vendors, Interviewees, Event Guests, and Recurring Visitors.
- The system shall support pre-approved visitor passes, OTP/PIN-based verification, QR-based entry passes, and time-limited digital passes.
- The system shall support multi-visitor entry grouping under a single approval/pass event.
- The system shall capture mandatory visitor photographs and vehicle numbers where applicable.
- The system shall screen incoming visitors against a blacklist and intercept blacklisted entries at the gate.
- The system shall support automated identification of frequent/recurring visitors.
- The system shall route visitor-approval decisions to the relevant resident via a mobile/web approval prompt and record the approve/reject decision.
- The system shall record gate-entry and gate-exit timestamps for every visit and retain the complete visitor lifecycle permanently in the audit trail.

FR-05 Security Gate Operations & Live Dashboard

- The system shall provide a high-speed, low-latency dashboard for security guards to manage live gate traffic.
- The system shall support instant visitor entry/exit processing, delivery verification, and cab/staff pass verification from the gate dashboard.
- The system shall support staff attendance stamping at the gate.
- The system shall allow guards to trigger emergency/panic alert broadcasts from the gate dashboard.
- The system shall intercept blacklisted entries before they are processed as normal gate transactions.
- The system shall allow the Security Supervisor to manage guard rosters, review incident logs, and override gate checkpoints.

FR-06 Domestic Staff & Multi-Apartment Assignment

- The system shall maintain profiles for maids, drivers, cooks, cleaners, babysitters, and gardeners, including biographic/ID details, photographs, police-verification status, and emergency addresses.
- The system shall support assigning a single staff member to multiple apartments within the same community.
- The system shall record live attendance and working hours for domestic staff at the gate.
- The system shall allow residents to submit ratings/feedback that is retained in the staff member's history.

FR-07 Delivery Management & Configurable Gate Protocols

- The system shall support delivery categories: Food, Grocery, E-commerce, Courier, and Pharmacy.
- The system shall support four configurable delivery protocols per resident/unit: Allow at Gate, Resident Approval Required, Leave at Gate Desk, and Direct Rejection.
- The system shall route the delivery to the correct protocol-driven workflow and record the resulting gate decision.
- The system shall maintain a complete audit log for every parcel, including protocol applied, decision, and timestamps.

FR-08 Vehicle, Parking & Violation Tracking

- The system shall maintain records for resident vehicles, visitor vehicles, cars, motorcycles, and service transport.
- The system shall support reserved parking slots and guest parking bays, with allocation and conflict checking.

- The system shall record automated vehicle entry/exit events at the gate.
- The system shall allow authorized users to report and track unauthorized parking as violation records.
- The system shall surface vehicle and parking status to authorized dashboards and reports.

FR-09 Accounting-Oriented Maintenance & Billing

- The system shall generate monthly maintenance invoices per flat-wise charge heads.
- The system shall support special infrastructure assessments, penalties, late fees, discounts, and advance payments.
- The system shall provide a simulated payment workflow suitable for sample/demo environments.
- The system shall generate receipts for recorded payments.
- The system shall maintain a resident ledger and produce real-time outstanding-balance reports.
- The system shall trigger maintenance-due notifications through the notification engine where dues are outstanding.

FR-10 Complaint & Service Desk (SLA-Driven)

- The system shall allow residents to raise service requests for plumbing, electrical, housekeeping, lifts, security, and common areas.
- The system shall enforce the ticket lifecycle: Created → Assigned → Acknowledged → In Progress → Resolved → Resident Confirmation → Closed.
- The system shall support priority levels and SLA timers per ticket, with configured escalation paths when an SLA is at risk or breached.
- The system shall support technician assignment and shall not permit a ticket to reach Closed status without resident confirmation.
- The system shall capture resident feedback ratings against the completed service record.

FR-11 Amenity Booking & Conflict Resolution

- The system shall allow residents to reserve clubhouses, gyms, swimming pools, tennis courts, community halls, and guest rooms.
- The system shall support slot management, capacity limits, and configurable booking rules.
- The system shall support maintenance blocks that render a slot unavailable for booking.
- The system shall perform a strict concurrency/conflict check before confirming any reservation, preventing overlapping bookings.
- The system shall reject or fail to confirm bookings that fail availability, capacity, or conflict checks, and shall return the conflict result to the requester.

FR-12 Community Communication & Broadcast Engine

- The system shall allow authorized admins to broadcast notices, emergency alerts, polls, surveys, and event-calendar items.
- The system shall allow a broadcast to be targeted to specific towers, blocks, resident groups, or an entire community.
- The system shall validate that the initiating admin is authorized to broadcast to the selected target scope before publishing.
- The system shall retain published broadcasts as part of the permanent community communication record.

FR-13 Emergency & Security Incident Management

- The system shall support logging of medical emergencies, fires, thefts, suspicious visitors, security breaches, and lift entrapments.
- The system shall record incident severity, location, and reporter details at creation.
- The system shall support assignment of responsible security personnel to an incident.
- The system shall record actions taken during incident response and the final resolution.
- The system shall retain a complete resolution audit trail for every incident for authorized compliance review.
- The system shall support triggering emergency-alert broadcasts/notifications from an active incident where applicable.

FR-14 Role-Specific Dashboards & KPI Summaries

- The system shall provide a Super Admin dashboard showing cross-community metrics: total residents, occupancy rates, active gate traffic, open complaints, and financial collections.
- The system shall provide a Community Admin dashboard showing community-specific KPIs, financial health, staff attendance, and incident logs.
- The system shall provide a Security dashboard showing live gate traffic, pending approvals, expected visitors, and emergency alerts.
- The system shall provide a Resident dashboard showing personal visitors, pending approvals, maintenance dues, service tickets, and notices.
- The system shall restrict every dashboard's underlying data to what the viewing user's role and community scope permit.

FR-15 Notification Engine & Integration Architecture

- The system shall provide an integration-ready notification framework supporting push notifications, with SMS and WhatsApp channels implemented as mocks.
- The system shall generate notifications for visitor approvals, maintenance-due alerts, and emergency notices at minimum.
- The system shall determine the correct recipient/audience for each notification from the originating business event and the recipient's access scope.
- The system shall treat the originating business record (visitor, invoice, incident, broadcast, etc.) as the authoritative source of truth; the notification itself is a derived communication event.

FR-16 Search, Filtering, Reporting & Audit Logging

- The system shall provide search and filtering across operational records relevant to each role's permitted scope.
- The system shall provide reporting capabilities required by the operational modules and role-specific dashboards.
- The system shall maintain immutable audit logs for all critical administrative operations.
- The system shall capture, at minimum, user ID, action performed, module, previous and new values, timestamp, and IP/session data in every audit record.
- The system shall apply role- and community-scope restrictions to all search, filter, and report results before returning them.

FR-17 Mandatory Seed & Synthetic Sample Data

- The system shall be delivered pre-populated with realistic synthetic data; empty screens are strictly prohibited.
- The delivered dataset shall include at least 2 communities, 4 towers, 8 floors, and 50 operational residential units.
- The delivered dataset shall include complete, representative distributions of owners, tenants, family members, security staff, domestic staff, visitors, vehicles, and parking slots.
- The delivered dataset shall include active maintenance invoices, payment records, service complaints, amenity bookings, notices, security incidents, and audit logs sufficient to demonstrate every module end-to-end.

FR-18 Technical Architecture & Security Requirements

- The backend shall be implemented in Python using FastAPI.
- The database shall be PostgreSQL, with enforced relational integrity and automated schema migrations.
- The frontend shall be implemented in Next.js with responsive CSS.
- The system shall use parameterized queries exclusively; construction of SQL via string concatenation of user input is prohibited.
- The system shall implement CSRF protection, rate limiting on sensitive endpoints, and server-side file-upload validation.
- The system shall sanitize all user-supplied input rendered back to the browser to prevent cross-site scripting (XSS).

FR-19 Weekly Task View, Sivion QA & Defect Lifecycle

- VPD shall submit a detailed weekly Task View tracking actual progress; verbal or estimated percentage reporting is not acceptable.
- Each Task View row shall record module, task name, assigned developer, planned start/end dates, status and percentage complete, testing/bug status, and Sivion status.
- Task completion by VPD's internal development team shall not, by itself, constitute task closure.
- Every task shall pass through the lifecycle: Development → VPD Internal Testing → Task Submission → Sivion Testing → Defect Report (if any) → VPD Fix → Sivion Retest → Final Acceptance.
- Final system acceptance shall require all required modules to operate end-to-end, synthetic sample data to be fully populated, RBAC to be independently verified, repository code to be inspected, and Sivion testing/retesting to be completed with zero unresolved Critical or High-severity defects.

4. External Interface Requirements

4.1 User Interfaces

- A responsive web UI built in Next.js, supporting role-specific views for all ten user classes.
- A dedicated, high-speed Security Gate Operations dashboard optimized for rapid, low-latency guard actions.
- Role-specific dashboards for Super Admin, Community Admin, Security, and Resident personas (Section 3, FR-14).
- Frontend components shall conditionally render controls based on the authenticated user's resolved role (RBAC-aware rendering).

4.2 Hardware Interfaces

No dedicated hardware interface (e.g., biometric devices, RFID readers, boom barriers) is specified in the source PRD. Gate operations are assumed to be performed by security personnel through the software dashboard rather than through integrated physical hardware, unless separately scoped.

4.3 Software Interfaces

- PostgreSQL relational database, accessed exclusively through parameterized queries/ORM.
- RESTful APIs exposed by the FastAPI backend, consumed by the Next.js frontend.
- An integration-ready notification framework with mocked SMS and WhatsApp channels (no specific third-party provider is mandated by the PRD).
- A simulated payment-processing workflow for the sample/demo environment (no live payment gateway is mandated by the PRD).

4.4 Communications Interfaces

- HTTPS for all client-server communication.
- Session/JWT-token-based authentication carried on each API request.
- Push-notification, SMS-mock, and WhatsApp-mock channels for outbound resident/staff communication.

5. Non-Functional Requirements

5.1 Security Requirements

- Secure password hashing using Argon2 or Bcrypt (NFR-SEC-01).
- Strict RBAC role isolation enforced at both API and frontend layers (NFR-SEC-02).
- Parameterized SQL queries to prevent SQL injection (NFR-SEC-03).
- XSS sanitization on all user-rendered content (NFR-SEC-04).
- CSRF protection on all state-changing requests (NFR-SEC-05).
- Rate limiting on sensitive endpoints (e.g., login, OTP verification, payment simulation) (NFR-SEC-06).
- File-upload validation (type, size, content) for all uploaded assets such as visitor photographs and staff ID documents (NFR-SEC-07).
- Strict multi-tenant data segregation between communities at the query/data-access layer (NFR-SEC-08).

5.2 Performance Requirements

- The Security Gate Operations dashboard shall be optimized for high-speed, low-latency interaction, since gate throughput directly affects physical wait times at the community entrance (NFR-PERF-01).

- Dashboard and reporting queries shall respect role/community scope filters without materially degrading response time as data volume grows across the minimum seed dataset (NFR-PERF-02).

5.3 Reliability & Availability

- Critical administrative operations shall be captured in immutable audit logs to support post-incident reliability review (NFR-REL-01).
- The amenity-booking engine shall guarantee that concurrency/conflict checks are atomic, so that no two overlapping reservations can both succeed (NFR-REL-02).

5.4 Maintainability

- Database schema changes shall be managed through automated, version-controlled migrations (NFR-MAINT-01).
- The codebase shall follow a micro-modular architecture aligned to the module boundaries defined in Section 3, to support independent testing and incremental delivery under the 30-day schedule (NFR-MAINT-02).

5.5 Compliance & Auditability

- Audit records shall be immutable once written and shall capture user ID, action, module, previous/new values, timestamp, and IP/session data (NFR-COMP-01).
- Auditors shall have read-only access to financial ledgers, audit logs, incident histories, and user-access reports, with no ability to modify underlying records (NFR-COMP-02).

6. Data Requirements — Mandatory Seed & Synthetic Sample Data

The delivered system must never present an empty screen. VPD must populate the environment with realistic synthetic data meeting the following minimums:

Entity	Minimum Requirement
Communities	At least 2 independent communities
Towers/Blocks	At least 4
Floors	At least 8
Residential Units	At least 50 operational units
Residents	Complete distribution of owners, tenants, and family members
Staff	Complete distribution of security staff and domestic staff
Vehicles & Parking	Representative vehicle records and parking-slot allocations
Financial Records	Active maintenance invoices and payment records
Service Records	Representative service complaints across all categories
Amenities	Representative amenity bookings across available amenities
Communication	Representative notices/broadcasts
Incidents	Representative security incidents with full lifecycle data
Audit Logs	Populated audit trail reflecting the above activity

7. Role-Based Access Control (RBAC) Matrix

GateSphere requires strict RBAC enforced at both the API routing layer and the frontend component-render layer. Unauthorized users must be entirely blocked from accessing restricted APIs, screens, and administrative records via URL manipulation.

Role	Primary Scope & Access Level	Administrative Capabilities
Super Admin	Global platform control	Manage multiple independent communities, global system settings, audit logs, and cross-community analytics.
Community Admin	Single community management	Manage community towers, staff, broadcast announcements, financial configurations, and resident approvals.
Association Committee	Governance & oversight	Access financial summaries, approve special assessments, review security incident reports, and audit collections.
Facility Manager	Operations & maintenance	Manage amenity bookings, facility maintenance schedules, vendor contracts, and common-area service desks.
Security Supervisor	Gate security oversight	Manage security guard rosters, blacklist visitors, review incident logs, and override gate checkpoints.
Security Guard	Live gate operations	Process visitor entries/exits, verify deliveries, check cab/staff passes, and trigger emergency panic alerts.
Owner / Tenant	Residential unit control	Approve visitors, pay maintenance bills, log complaints, book amenities, and manage family/vehicle records.
Domestic Staff	Workforce access	View assigned apartments, check in via gate pass, and view personal attendance/ratings.
Vendor / Technician	Service execution	Update assigned service tickets, log entry passes, and submit work-completion proofs.
Auditor	Read-only compliance	Access financial ledgers, system audit logs, incident histories, and user-access reports without edit privileges.

8. Quality Assurance, Defect Classification & Acceptance Criteria

8.1 Sivion Independent Testing Workflow

Sivion Global Technologies acts as the independent testing and validation authority. Development completion by VPD does not, by itself, constitute task closure. Every task follows the required lifecycle:

- 1 Development
- 2 VPD Internal Testing
- 3 Task Submission
- 4 Sivion Testing
- 5 Defect Report (if applicable)
- 6 VPD Fix
- 7 Sivion Retest
- 8 Final Acceptance

8.2 Defect Classification

Severity	Definition	Acceptance Implication
Critical	Security breach, data corruption, authentication failure, or business-critical outage.	Must be 100% resolved before acceptance.
High	Major functionality unavailable without a reasonable workaround.	Must be 100% resolved before acceptance.
Medium / Low	Minor UI or usability imperfections.	Tracked and remediated according to the standard QA/acceptance process; not blocking for final sign-off.

8.3 Final Acceptance Conditions

The GateSphere project shall be deemed ready for acceptance only when all of the following conditions are simultaneously satisfied:

- All required modules operate end-to-end as specified in Section 3.
- Synthetic sample data is fully populated per Section 6.
- RBAC is independently verified per the matrix in Section 7.
- Repository code has been inspected.
- Sivion testing and retesting have been successfully completed with zero unresolved Critical or High-severity defects.

8.4 Weekly Task View

VPD must submit a detailed weekly Task View tracking actual (not verbal or estimated) progress. Each row must record: Module, Task Name, Assigned Developer, Planned Start/End dates, Status & Percentage complete, Testing & Bugs, and Sivion Status.

9. Delivery Schedule & Change-Control Protocol

9.1 Mandatory Delivery Schedule — Week-Wise Timeline

The complete engineering lifecycle — design, development, testing validation, bug remediation, and final handover — must be executed strictly within **30 calendar days** of the officially approved project start date. This 30-calendar-day period is the binding contractual ceiling and is broken down below into a week-wise timeline for planning, tracking, and weekly Task View reporting.

Week	Calendar Window	Primary Focus & Scope
Week 1	Calendar Days 1–7	Architecture finalization; ERD design; UI/UX system setup (Next.js); authentication & JWT/session framework (FastAPI); RBAC framework across all ten roles; community, tower, floor, unit & resident master-data structure; foundational REST APIs.
Week 2	Calendar Days 8–14	Visitor management lifecycle (categories, passes, OTP/QR, approvals, blacklist); security guard live gate dashboard; domestic staff management & multi-apartment assignment; delivery workflows & configurable gate protocols; vehicle association & parking allocation; notification-engine foundation.
Week 3	Calendar Days 15–21	Maintenance invoicing & accounting ledgers; simulated payment workflow; complaint/service-desk module with SLA timers & escalation; amenity booking conflict/concurrency engine; notices, broadcasts & announcements; emergency & security incident management; reporting modules.
Week 4	Calendar Days 22–28	Role-specific dashboard completion (Super Admin, Community Admin, Security, Resident); immutable audit logging across all modules; synthetic seed-data injection (Section 6 minimums); end-to-end workflow verification; security hardening (CSRF, rate limiting, input sanitization, file-upload validation).
Final Buffer	Calendar Days 29–30	Sivion QA validation pass, defect triage and closure of any remaining Critical/High-severity defects, Sivion retest, repository code inspection, and final handover — within the strict 30-calendar-day ceiling.

Each week's deliverables map directly to the corresponding functional requirements in Section 3 (FR-01...FR-19) and shall be reported against in that week's Task View (Section 8.4). Weekly progress must reflect actual, verifiable completion status — not verbal or estimated percentages — and each completed task must pass through the full Sivion QA lifecycle defined in Section 8.1 before being marked closed in the Task View, regardless of which week it was originally scheduled in.

9.2 Extension & Delay Protocol

If VPD determines that the defined scope cannot be fully delivered within 30 calendar days, VPD must NOT begin development and subsequently request an extension. Before project kickoff, VPD must submit a formal email detailing:

- Technical justifications for the anticipated delay,
- The specific modules causing delays,
- Proposed revised delivery dates,
- Resource allocations, and
- Schedule impact.

Development may proceed on a revised timeline only upon written approval. Starting development without such notice binds VPD to the strict 30-calendar-day delivery commitment.

10. Traceability & Interpretation Notes

- This SRS is derived from the supplied GateSphere PRD and its accompanying module-workflow interpretation; both source documents were compared and found to contain no substantive requirement differences.
- Where the PRD names a capability without specifying its exact UI/API implementation, the implementation team retains latitude to choose the concrete technical design while preserving the stated business behavior.
- This document does not introduce requirements beyond the PRD's stated scope — in particular, it does not add live payment-gateway integration, a specific SMS/WhatsApp provider, biometric attendance, facial recognition, GPS tracking, or external police-verification APIs, none of which are specified in the source PRD.
- Each functional requirement (FR-01...FR-19) in Section 3 is intended to map directly to a line item in the Weekly Task View (Section 8.4) and to a corresponding Sivion test case.
- The technology stack referenced throughout this document is standardized as: **Frontend — Next.js; Backend — FastAPI (Python); Database — PostgreSQL**. No other frontend or backend framework is in scope for this build.